

L-2 项目 · 前端对接文档

用途:供前端同学对接后端 API。涵盖启动、跨域、所有端点、关键字段语义、典型对接流程、错误处理、避坑提示。 **版本:**2026-05-25 **后端版本:**see git log;Swagger 文档自动同步,以 `/docs` 为准

本文档 + Swagger UI(`/docs`) 一起用最香:本文档讲业务语义和陷阱,Swagger 给字段定义和实时调试。

一、启动后端

```
cd /path/to/learning-path
.venv/bin/uvicorn app.main:app --port 8000 --reload
```

后端默认监听 `http://localhost:8000`。

健康检查:

```
curl http://localhost:8000/health
# {"status":"ok"}
```

如本机第一次跑,请先确认: - `.env` 里有 `ARK_API_KEY` (否则 LLM 走 mock 兜底) - 已跑过 `python -m scripts.seed_concepts` 与 `python -m scripts.import_questions <xlsx>` - 已跑过 `python -m scripts.init_rag` (可选,不跑则 RAG 不命中,LLM 仍能工作)

二、CORS 与跨域

后端已默认允许以下 origin:

- `http://localhost:5173` (Vite dev 默认端口)
- `http://127.0.0.1:5173`

前端开发期推荐 vite proxy 而非直连(避免 preflight 偶尔抽风):

```
// vite.config.js
export default {
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:8000',
        changeOrigin: true,
        rewrite: (p) => p.replace(/^\/api/, ''),
      },
    },
  },
}
```

前端调用 `fetch('/api/profile')` ,生产部署时用同源直连(后端 serve 静态文件)。

三、所有端点一览

Method	Path	说明	耗时
GET	<code>/health</code>	健康检查	< 50ms
POST	<code>/profile</code>	创建学生画像	< 100ms
GET	<code>/profile/{id}</code>	取单个学生	< 100ms
GET	<code>/profile</code>	列出所有学生(按创建倒序)	< 200ms
GET	<code>/concepts?subject=管综数学</code>	列出知识点(33 个)	< 100ms
GET	<code>/questions/sample?n=5</code>	从题库抽 N 题(每个 concept $\leq$ 1 道)	< 200ms
POST	<code>/diagnose</code>	提交答卷 $\rightarrow$ LLM 诊断 + 生成路径	5-15 s
GET	<code>/path/{student_id}</code>	取学生当前学习路径(db 持久化)	< 100ms
POST	<code>/interaction</code>	提交学习交互事件 $\rightarrow$ LLM 优化路径	5-15 s

⚠ `/diagnose` 、 `/interaction` 走 LLM,前端必须做 **loading 状态**,否则用户以为卡死。

四、详细端点说明

4.1 POST /profile

用途:创建学生画像。

请求体(`Content-Type: application/json`):

```
{
  "nickname": "演示同学",
  "subject": "管综数学",
  "cognitive_level": "intermediate",
  "learning_goal": "2027 年管综数学过国家线",
  "available_minutes_per_day": 60,
  "learning_style": "reading"
}
```

字段约束:

字段	类型	必填	约束
nickname	str	√	1-50 字
subject	str	√	1-50 字,目前固定填 "管综数学"
cognitive_level	enum	√	<code>beginner</code> / <code>intermediate</code> / <code>advanced</code>
learning_goal	str	√	自由文本,>= 1
available_minutes_per_day	int	√	5-600
learning_style	enum	√	<code>visual</code> / <code>auditory</code> / <code>kinesthetic</code> / <code>reading</code>

响应 201:

```
{
  "id": 4,
  "nickname": "演示同学",
  "subject": "管综数学",
  "cognitive_level": "intermediate",
  "learning_goal": "...",
  "available_minutes_per_day": 60,
  "learning_style": "reading",
  "created_at": "2026-05-25T08:30:12",
  "updated_at": "2026-05-25T08:30:12"
}
```

4.2 GET /concepts

用途:获取知识图谱节点列表(供下拉选择 / 知识图谱可视化)。

Query: - `subject` (可选):学科过滤,目前传 `"管综数学"`

示例:

```
GET /concepts?subject=管综数学
```

响应 200,返回 33 个节点(按 code 排序):

```
[
  {
    "code": "代数-数列-等差",
    "name": "等差数列",
    "subject": "管综数学",
    "prerequisite_codes": ["代数-整式运算"],
    "description": "通项公式、求和公式、性质"
  },
  ...
]
```

`prerequisite_codes` 是当前节点的先修依赖,可用来画知识图谱箭头。

4.3 GET /questions/sample

用途:抽题(每个 concept_code 至多 1 道,保证多样性)。

Query: - `n` (可选):抽题数量,1-30,默认 5

响应 200:

```
[
  {
    "id": 156,
    "concept_code": "代数-数列-等差",
    "question_type": "single_choice",
    "stem": "已知等差数列  $\{a_n\}$  首项  $a_1=2$ , 公差  $d=3$ , 则  $a_{10}=?$ ",
    "choices": {"A": "27", "B": "29", "C": "30", "D": "31", "E": "32"},
    "answer": "B",
    "difficulty": 2,
    "score": 3,
    "explanation": " $a_{10}=a_1+(10-1)d=2+9\times 3=29$ ",
    "source": "2024真题",
    "year": 2024
  },
  ...
]
```

⚠ 关键: `stem` / `choices.*` / `explanation` 含 LaTeX 原文(`$...$` 或 `\dfrac{...}{...}` 之类),前端必须用 **KaTeX 渲染**(见 § 六)。

4.4 POST /diagnose

用途:接收学生答卷,LLM 诊断每知识点掌握度并生成首版路径。

请求体:

```
{
  "student_id": 4,
  "answers": [
    {
      "question_id": 156,
      "concept_id": "代数-数列-等差",
      "correct": false,
      "seconds": 90
    },
    {
      "question_id": 157,
      "concept_id": "代数-数列-等差",
      "correct": true,
      "seconds": 30
    }
  ]
}
```

`answers` 至少 1 项; `concept_id` 取自 question 的 `concept_code`。

响应 200:

```
{
  "student_id": 4,
  "mastery": {
    "代数-数列-等差": 0.45,
    "代数-数列-等比": 0.10
  },
  "path": [
    {
      "concept_id": "代数-数列-等差",
      "title": "等差数列通项与求和速记",
      "estimated_minutes": 15,
      "reason": "你对等差数列掌握度较低,优先夯实公式..."
    }
  ],
  "reasoning": [
    "[diagnose] LLM: 等差掌握不牢,等比基本不会",
    "[plan] RAG 检索命中 1 条候选资源",
    "[plan] LLM: 按掌握度从低到高..."
  ],
  "mock": false
}
```

字段语义:

字段	说明
mastery	知识点掌握度,值 0-1。0.4 以下建议红色,0.7 以上绿色
path	推荐学习路径,3-5 项
path[].concept_id	⚠ 注意此处叫 concept_id ,不是 concept_code (同一种 ID,字段名不同,见 § 六)
reasoning	推理日志(给老师 / 调试用,前端可折叠展示)
mock	false = LLM 真调用了; true = LLM 不可用走了 mock 兜底

4.5 GET /path/{student_id}

用途:取该学生当前学习路径(db 持久化,uvicorn 重启不丢)。

响应 200:

```
{
  "student_id": 4,
  "path": [...],
  "mock": false
}
```

响应 404(没跑过 /diagnose):

```
{ "detail": "no path; run /diagnose first" }
```

4.6 POST /interaction

用途:学生在学习中遇到问题或已掌握,反馈给 LLM 优化路径。

请求体:

```
{
  "student_id": 4,
  "event": "struggle",
  "concept_id": "代数-数列-等差",
  "detail": "公式记不住"
}
```

字段:

字段	取值	说明
event	struggle / mastered / skip	卡住 / 已掌握 / 暂时跳过
concept_id	33 个 concept_code 之一	来自 /concepts
detail	str / null	自由文本,LLM 会读

响应 200(同 /diagnose ,但只含 path/reasoning/mock,无 mastery):


```
{
  "student_id": 4,
  "path": [...新路径...],
  "reasoning": [
    "[optimize] LLM: 在等差节点后插入针对性补强练习"
  ],
  "mock": false
}
```

前端可对比新旧 path,对**新增项**做高亮或动画(`title` 是稳定可比的字段)。

五、典型对接流程

- ① POST /profile → 拿到 student_id
前端把 student_id 存 store
- ② GET /questions/sample?n=5
前端渲染题目,等学生答题
- ③ (前端模拟答题,逐题点对/错)
- ④ POST /diagnose → 显示 mastery 条 + path 列表
LLM 调用 5-15s,务必 loading
- ⑤ GET /concepts → 学习交互页面的下拉选项
- ⑥ POST /interaction → 拿到新 path,前端对比旧 path 做新增项高亮
可重复触发(每次学习反馈)

六、关键陷阱(必看)

6.1 `concept_id` VS `concept_code`

后端两套字段名指向同一种 ID:

场景	字段名
<code>Concept</code> 表 / <code>Question</code> 表 / <code>concepts</code> API	<code>concept_code</code>
<code>PathItem</code> (/diagnose、/path、/interaction 返回的 path 项)	<code>concept_id</code>
<code>/interaction</code> 请求体	<code>concept_id</code>
<code>/diagnose</code> 请求体 <code>answers[].concept_id</code>	<code>concept_id</code>

⚠ 历史包袱,后端不打算改(改了破坏现有调用方)。前端记住 `path` 项里读 `concept_id` 即可。

6.2 LaTeX 原文渲染

`stem` / `choices` 值 / `explanation` 字段是 LaTeX 原文,可能含:

- 行内公式 `$a_1 = 2$`
- 块公式 `$$\sum_{i=1}^n a_i$$`
- 反斜杠命令 `\dfrac{1}{2}`、`\sqrt{x}`、`\pi`

推荐 KaTeX:

```
npm install katex
```

```
import 'katex/dist/katex.min.css'
import renderMathInElement from 'katex/dist/contrib/auto-render.mjs'

// 渲染时机: nextTick 后,确保 DOM 已挂载
renderMathInElement(el, {
  delimiters: [
    { left: '$$', right: '$$', display: true },
    { left: '$', right: '$', display: false },
  ],
  throwOnError: false,
})
```

⚠ 不要做 HTML 转义(否则 LaTeX 命令失效)。直接用 `v-html` 或 `textContent` 即可,LaTeX 自带的反斜杠对 XSS 无威胁。

6.3 `mock` 字段

- `false` = 真 LLM(火山方舟 Doubao)调用成功
- `true` = LLM 不可用(网络 / API key 缺失 / 超时),走了本地启发式 mock

前端可显示徽章  /  让老师演示时一眼看出。

6.4 LLM 调用耗时

`/diagnose`、`/interaction` 实测 **5-15 秒**(国内访问火山方舟)。前端必须:

- 显示 spinner / loading 文案
- 禁用按钮防重复提交
- 不要设置过短超时(< 30s 都太短)

6.5 重启后路径不丢

`/path/{id}` 从 `learning_paths` 表取,uvicorn 重启不丢。但**进程内 student state 不应缓存太多**,前端可信任 `/path` 总是当前最新。

七、错误处理

所有 4xx / 5xx 返回:

```
{ "detail": "..." }
```

常见:

状态	场景	应对
400	入参校验失败(Pydantic 自动)	显示 <code>detail</code> (很详细)
404	<code>student not found</code> / <code>no path; run /diagnose first</code>	引导用户先做上一步
422	请求体反序列化失败	检查 JSON 格式
500	后端 bug(理论上 LLM 异常已被 mock 兜底)	截图给后端同学

八、参考资源

资源	说明
Swagger UI: http://localhost:8000/docs	字段实时同步,可点击 "Try it out" 直接调试
OpenAPI JSON: http://localhost:8000/openapi.json	可用 <code>openapi-typescript</code> 生成 TS 类型
<code>frontend/</code> (Vue 工程)	AI 第一版参考实现,可借鉴 store / api 封装,UI 风格请重做
<code>web/demo.html</code>	单文件原生 HTML 实现,可作 fallback
<code>docs/概念清单.pdf</code>	33 个 concept_code 与 (category, knowledge_point) 对照表

九、后端联系人

- 接口字段、错误码、CORS、性能问题:找张同学(后端)
- 题库 / 资源 / 概念清单录入:找[录题同学]
- 测试用账号(前端调试用):跑 `POST /profile` 即可创建

如有"`mock=true` 但我已经设了 ARK_API_KEY"之类的怪事,优先看 uvicorn 控制台日志(里面会打印 `LLM call failed: ...` 原因)。